

Business Case — Seller Acquisition ROI Under Uncertainty

A learning project built to look like client work.

1. Context

Olist runs a Brazilian marketplace and grows it by acquiring third-party sellers. Two teams commit independently:

- **Marketing/Sales** commits a channel budget and SDR/SR headcount, expecting a volume of qualified leads and closed deals.
- **Marketplace/Finance** commits to the sellers that close, expecting them to generate GMV (and Olist's commission on it) for years to come.

Neither owns the number that matters — **return on acquisition spend** — and neither knows how sensitive it is to the other's assumptions. Marketing pays the acquisition cost the day a deal closes. Revenue, if it comes at all, arrives as a thin, decaying monthly trickle conditioned on the seller actually listing products, getting orders, and not churning.

2. The decision this supports

"Which acquisition channels and lead segments should get next quarter's marketing budget, and how much seller lifetime value can we defensibly expect per euro of CAC, given how uncertain that link is?"

The output is not a lead-volume forecast. It is a **channel-level LTV:CAC distribution plus a ranked list of which assumption to go and verify first.**

3. Data strategy — two datasets, one real join, mostly missing

This is the central design decision, and it is deliberate.

Marketing Funnel (MQL + Closed Deals)		Brazilian E-Commerce (orders + 7 siblings)
Role	Acquisition source	Outcome source
Contributes	Channel, lead quality, time-to-close	GMV, order volume, review score, seller lifespan
Grain used	lead × day → channel × week	order item × day → seller × month
Volume	8,000 leads, 842 closed deals	99,441 orders, 112,650 order items, 3,095 sellers
Period	Jun 2017 – May 2018 (leads), Dec 2017 – Nov 2018 (deals)	Sep 2016 – Oct 2018

Unlike the previous version of this project (which deliberately never joined its two sources because no real key connected them), this time the join key is genuine: `closed_deals.seller_id` is the same `seller_id` used throughout the e-commerce dataset. Real, but far from complete — and that incompleteness is the actual lesson.

Two funnel leaks, verified against the raw data before writing a line of dbt:

1. **8,000 MQLs → 842 closed deals (10.5%).** Expected — most leads don't close.
2. **842 closed deals → 380 sellers with any order at all (45%).** A deal marked "won" is not the same as a seller who ever shipped a product. Roughly half of closed deals never show up as revenue.
3. **Of the 3,095 sellers with at least one order, only 380 (12%) carry any acquisition record at all.** MQL tracking starts June 2017; the marketplace itself starts September 2016. Most of the historical seller base predates the funnel being tracked.

The discipline this forces: the 88% of sellers with no tracked channel are not dropped and not assigned a fabricated one — they're flagged `est_is_attributed = false` and carried through every downstream mart. Any channel-level ROI number is honestly scoped to the attributed 12%, and the marts say so in their own schema rather than in a footnote someone can miss.

The third source is synthesis, same as before. Olist doesn't publish marketing spend, SDR cost, or its own take rate. Cost-per-lead, SDR hours-per-close, and the commission rate are declared parameters anchored on published B2B/marketplace CAC benchmarks, not measured facts — `est_`-prefixed, same convention as last time, same reason: making assumptions visible and testable is the actual deliverable here, not a compromise forced by open data.

4. Scope

In scope: channel × week grain for the funnel, seller × month grain for GMV, 13-week (one-quarter) forward LTV projection per acquisition cohort, deterministic scenarios plus Monte Carlo on CAC/churn/close-rate assumptions, one dashboard view.

Out of scope: individual lead scoring or propensity models, multi-touch attribution (only `origin`, a last-touch field, is available), real-time bid optimization, pricing.

5. Success criteria

Criterion	Target
GMV-ramp forecast accuracy vs cohort-naive baseline	WAPE improvement $\geq 15\%$
Backtest coverage	P10–P90 ROI interval contains actuals $\geq 80\%$ of cohort-months
Reproducibility	Full rebuild from empty cloud project in < 30 min
Portability	Same dbt models run on DuckDB and BigQuery without edits
Pipeline reliability	4 consecutive scheduled runs with no manual fix
Explainability	Every ROI number traceable to a declared assumption, including attribution coverage

6. Cost, and a declared assumption about scale

€0 expected, same as before — both datasets fit inside BigQuery's free tier by a wide margin.

The combined raw CSVs are small: the e-commerce tables (orders, items, payments, reviews, customers, sellers, products, geolocation) total **~126 MB**, and the funnel tables add **~0.9 MB**. **geolocation** alone (1,000,163 rows, ~61 MB) is the one table with real row-count heft — everything else is a few hundred thousand rows at most.

Declared assumption: this project treats the public Kaggle export as a representative sample of a production system running at multi-GB scale — geolocation pings, funnel touchpoints, and order events at a real marketplace's actual volume, of which Kaggle's export is a bounded slice. The pipeline is built as if it ingests that full volume: partitioning, clustering, and the BigQuery-vs-DuckDB cost comparisons in Week 2 (**tutorial.md**) are exercised against the real files using the same discipline (partition pruning, **maximum_bytes_billed**, **SELECT** fewer columns) that a multi-GB production table would demand. This is stated plainly here, in the schema (nothing pretends to be a bigger table than it is), and again wherever a tutorial step's *framing* — not its data — assumes scale. No rows are synthetically inflated by default.

The residual risk is the same as before: a careless unpartitioned scan repeated in a loop. The design prevents it, and **maximum_bytes_billed** enforces it.

7. Risks and limitations

Stated up front, because a model that hides these is worse than no model.

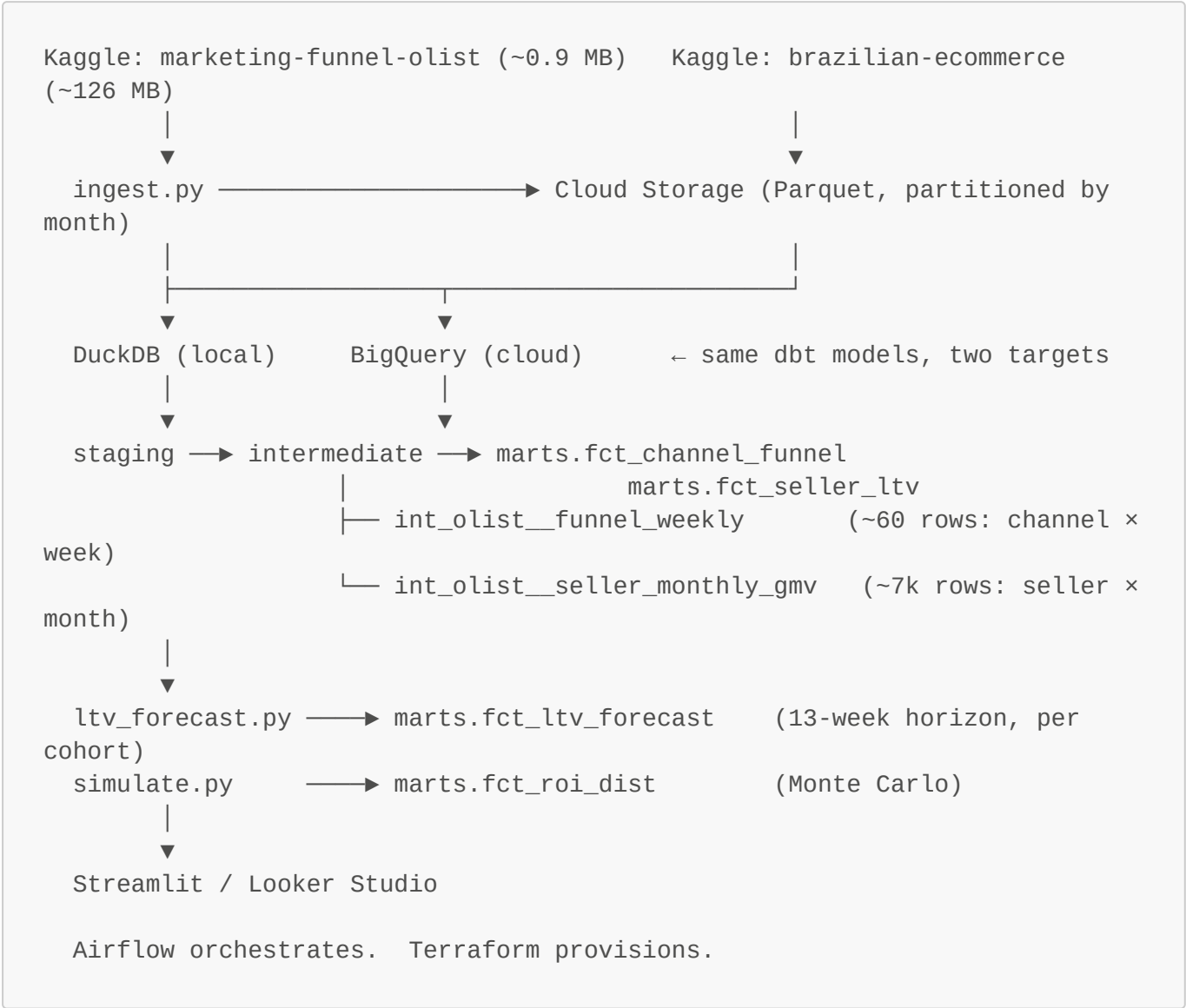
- **Attribution covers 12% of sellers.** Every channel-level ROI conclusion generalizes only to the attributed subset. The unattributed 88% still counts in total marketplace GMV — it's excluded from channel comparisons, not from the business.
- **CAC is synthesized.** Cost-per-lead and SDR cost are modeled parameters, not measured facts. Every ROI figure is conditional on them. **est_** prefixes say which numbers are real.
- **Correlation, not causation.** **origin** correlates with deal quality. Leads aren't randomly assigned to channels or SDRs — a channel that looks better may just be getting better-routed leads.
- **Thin cells.** 842 closed deals across 34 business segments means most segment-level cuts are single-digit counts. Channel, not segment, is the safe primary grain.
- **A won deal isn't a selling seller.** 462 of 842 closed deals (55%) never produced an order in this snapshot — timing, or genuine no-shows. Treated as a second funnel stage, not noise to filter out.
- **Dataset era and geography.** Sep 2016 – Oct 2018, Brazil. Absolute CAC and GMV levels are historical and local; the funnel mechanics and the leak structure transfer.
- **Scale is a declared assumption, not a measurement** (§6). Anyone re-running the cost-comparison exercises against the real files will see genuinely small numbers; the multi-GB framing is pedagogical, and this document says so rather than letting a reader discover it.

8. Timeline

Six weeks at 1–2h/day. Weeks 1–3 build the pipeline; weeks 4–6 build the forecast, the scenario engine, and the presentation layer.

Technical Appendix

A. Architecture



B. Data model

Two grains, joined only where the funnel record actually exists: **channel × ISO week** (funnel side) and **seller × calendar month** (outcome side).

Layer	Model	Contents
staging	stg_olist__mql, __closed_deals	typed, renamed, deduplicated
staging	stg_olist__orders, __order_items, __order_payments, __order_reviews, __sellers, __products, __customers	typed, renamed, deduplicated
intermediate	int_olist__funnel_weekly	MQL count, closed-deal count, conversion rate, by origin × week

Layer	Model	Contents
intermediate	<code>int_olist__seller_monthly_gmv</code>	GMV, order count, freight, avg review score, by seller × month
intermediate	<code>int_olist__seller_acquisition</code>	origin, lead_type, business_type, sdr_id, days-to-close — one row per attributed seller (380 rows)
marts	<code>fct_channel_funnel</code>	historical weekly funnel volume and conversion, by origin
marts	<code>fct_seller_ltv</code>	seller-month GMV joined to acquisition where it exists, <code>est_is_attributed</code> flag
marts	<code>fct_channel_roi</code>	channel-cohort CAC vs. cumulative take-rate revenue to date
marts	<code>fct_ltv_forecast</code>	predicted seller-month GMV, 13-week horizon
marts	<code>fct_roi_scenario_grid</code>	forecast × scenario parameters (cross join)
marts	<code>fct_roi_dist</code>	Monte Carlo output, P10/P50/P90 portfolio ROI
seeds	<code>scenario_parameters.csv</code>	the assumption set — the heart of the project
seeds	<code>channel_cost_benchmarks.csv</code>	synthesized cost-per-lead and SDR hours per close, by origin, with sources

The two staging groups meet at exactly one key, `seller_id`, and only for the 380 sellers where it resolves. Everywhere else, `fct_seller_ltv.est_is_attributed = false` and the row still counts toward total GMV, just not toward any channel's ROI.

C. The acquisition ROI model

For channel c in cohort month m :

```

leads_c,m      = observed MQL count, origin = c
close_rate_c   = closed_deals_c / leads_c           ← history;
scenario can multiply
sellers_won_c,m = leads_c,m × close_rate_c

gmv_per_seller_t = forecast(monthly GMV | months since won_date, channel
cohort) ← seller ramp curve
est_take_revenue_t = gmv_per_seller_t × take_rate

```

```

est_cac_c          = est_cost_per_lead_c × (leads_c / closed_deals_c)
                   + sdr_cost_per_hour × est_sdr_hours_per_close_c

est_ltv_c          = Σ_t  est_take_revenue_t × retention_curve(t)

est_roi_c          = (est_ltv_c - est_cac_c) / est_cac_c
payback_months_c   = first t where cumsum(est_take_revenue_t) ≥ est_cac_c

```

`take_rate`, `retention_curve`, and the cost benchmarks come from the seeds. `close_rate_c` and `gmv_per_seller_t`'s baseline shape are derived from `int_olist__funnel_weekly` and `int_olist__seller_monthly_gmv`. `sellers_won_c, m` is a scenario-driven projection; `gmv_per_seller_t` is fit against the 380 attributed sellers' actual post-close ramp and applied to the projected cohort.

Note the asymmetry, which is the single most interesting output: **CAC is paid in full the day a deal closes; revenue is a conditional, decaying stream that starts only if the seller ever lists and lasts only as long as they don't churn.** A close-rate-only view of channel performance cannot see this, and it makes the downside tail — a channel with a great close rate feeding sellers who never ship — materially fatter than a naive CAC-per-lead comparison would suggest.

D. Sensitivity analysis

Three levels, increasing in rigour:

1. **One-at-a-time (OAT)** — vary each parameter across its plausible range, hold others at base. Output: a tornado chart ranking drivers by ROI swing. Cheap, and it answers "what do I verify first?"
2. **Scenario grid** — named combinations (base / pessimistic / optimistic / *SDR capacity crunch* / *CAC inflation* / *retention shock*). Implemented in dbt as a cross join between the cohort forecast and the seed, so scenarios are version-controlled data rather than code.
3. **Monte Carlo** — 10,000 draws, one distribution per parameter (triangular where you only know min/mode/max, which is most of them). Output: full portfolio-ROI distribution.

Because the synthesized parameters (CAC, retention curve) carry more uncertainty than the measured ones (lead volume, historical close rate), their ranges should be *wider*. Expect the tornado chart to rank `retention_curve` and `est_cost_per_lead` above anything derived from real funnel counts. That result is honest and worth stating plainly: the model's biggest lever is the thing you know least about, which is exactly the finding that sends someone to go get better data — or, here, to go find out what Olist's actual take rate and churn curve really are.

Stretch: Sobol variance decomposition via SALib. OAT misleads when parameters interact; Sobol attributes variance including interaction terms.

E. Forecasting approach

Deliberately boring, in this order:

1. **Cohort-naive baseline.** A seller's GMV in month t since close equals the attributed cohort's observed median GMV at month t . This is the bar, and reporting against it is what makes the result credible.

2. **Statistical model.** A retention/decay curve (exponential or Weibull survival) fit on the 380 attributed sellers' actual post-close trajectories — appropriate given how few cohort-months of real data exist.
3. **Gradient boosting**, only if 1 and 2 are exhausted: seller age, channel, lead_type, business_type, declared catalog size, month-of-year. LightGBM.

The genuine constraint here, unlike the previous version of this project, isn't data volume — it's cohort depth. 380 attributed sellers over at most ~11 months of tracked history is not much to fit a decay curve on, and the forecast should say so rather than overstate its own precision.

Evaluation: rolling-origin backtest by cohort month, WAPE headline. Never a single time split.

F. Stack

Python 3.14 · uv · Docker · DuckDB · Google Cloud Storage · BigQuery · dbt Core (**dbt-duckdb** + **dbt-bigquery**) · Airflow 3 · Terraform · statsmodels / LightGBM · SALib · Streamlit · GitHub Actions · Kaggle API

G. On the GPU

The compute profile is tens of MB of tabular data, a few hundred cohort-rows, tabular models. **A GPU contributes nothing.** Two defensible uses, both optional: Monte Carlo at 10M draws via CuPy, and **XGBoost(device="cuda")** as a one-line change. Neither belongs in the critical path.